

PredictX

Security Analysis & Threat Model

Version 1.0 | March 2025 | Sepolia Testnet

⚠ This document is a self-conducted security analysis, not a formal third-party audit.

Disclaimer

This document represents a self-conducted security analysis of the PredictX smart contract system. It is NOT a formal third-party security audit. The contracts have NOT been audited by a professional security firm.

PredictX is deployed on the Sepolia testnet for demonstration and portfolio purposes only. It should NOT be used with real mainnet funds until a formal audit is completed by a qualified security firm.

1. Scope of Analysis

1.1 Contracts Analyzed

Contract	Lines of Code	Risk Level
MarketFactory.sol	~200	HIGH — handles child contract deployment
PredictionMarket.sol	~280	CRITICAL — holds user ETH
MultiOutcomeMarket.sol	~220	CRITICAL — holds user ETH
MultiMarketFactory.sol	~180	HIGH — handles child contract deployment
PREDToken.sol	~80	MEDIUM — ERC20 token
LiquidityMining.sol	~120	MEDIUM — controls PRED minting
ReferralSystem.sol	~130	MEDIUM — holds ETH temporarily
ChainlinkOracle.sol	~140	LOW — read-only price feed wrapper
MultiOracle.sol	~80	MEDIUM — controls resolution

1.2 Analysis Methodology

- Manual code review of all contract logic
- Review against known Solidity vulnerability patterns (SWC Registry)
- Trace of all ETH flow paths (deposit, hold, transfer)
- Review of access control patterns on all privileged functions
- Analysis of oracle integration for manipulation vectors
- Review of reentrancy attack surfaces

2. Threat Model

2.1 Assets at Risk

Asset	Location	Risk
User ETH (trade deposits)	PredictionMarket.sol, MultiOutcomeMarket.sol	CRITICAL
Protocol fees	feeCollector address	MEDIUM
PRED token mint authority	LiquidityMining.sol minter role	HIGH
Referral earnings	ReferralSystem.sol	MEDIUM
Market resolution outcome	Oracle contracts	HIGH
Admin control	Owner addresses	HIGH

2.2 Threat Actors

Actor	Capability	Goal
External attacker	Deploy contracts, call any public function	Drain ETH from markets
Malicious market creator	Create markets, set parameters	Manipulate resolution outcome
Compromised admin	Owner-level access to all contracts	Drain fees, change oracle, mint PRED
Oracle manipulator	Control resolution outcome	Force incorrect resolution, steal pool
Front-runner	Observe pending transactions	Extract MEV from large trades

3. Vulnerability Analysis

3.1 Complete Threat Register

The following table lists all identified threats, their severity, and implemented mitigations:

Severity	Threat	Location	Mitigation
CRITICAL	Reentrancy on claimReward()	PredictionMarket.sol:115	ReentrancyGuard modifier applied. hasClaimed flag set before ETH transfer (CEI pattern).
CRITICAL	Reentrancy on buyYesShares/buyNoShares	PredictionMarket.sol:82,93	ReentrancyGuard on all payable functions. Fee sent after pool update.
HIGH	Admin key compromise	All contracts (owner)	Single owner pattern. Recommendation: use multisig (Gnosis Safe) for mainnet.
HIGH	Oracle manipulation — forced incorrect resolution	ChainlinkOracle.sol, MockOracle.sol	MockOracle restricted to admin. Chainlink uses tamper-resistant aggregated price feeds.
HIGH	Market creation with past expiry	MarketFactory.sol:52	Block timestamp check in createMarket(). 300-second buffer added in frontend.
HIGH	Stack too deep in constructor	MultiOutcomeMarket.sol	Fixed by splitting getMarketInfo() into two functions and enabling vialR in foundry.toml.
MEDIUM	Fee griefing — fee call failure	PredictionMarket.sol	Fee transfer uses low-level call with success check. Reverts if fee transfer fails.
MEDIUM	PRED inflation via authorized caller abuse	LiquidityMining.sol	setAuthorizedCaller() restricted to owner. Only factory addresses are authorized.
MEDIUM	Referral self-referral	ReferralSystem.sol	cannotReferSelf() check implemented.
MEDIUM	Front-running market resolution	All market contracts	Resolution is callable by anyone after expiry.

MEDIUM			Front-running doesn't change outcome.
	Empty market resolution (zero pool)	PredictionMarket.sol	INVALID outcome handles zero-pool edge case with proportional refund.
LOW	Subgraph category empty for old markets	predictx/src/mapping.ts	Fixed in v0.0.3 — reads category via contract call at index time.
LOW	Gas estimation failure for createMarket	MarketFactory.sol, frontend	Fixed by hardcoding gas: BigInt(3000000) in frontend transaction.
LOW	ChunkLoadError from browser cache	Next.js frontend	Fixed by always testing in incognito and running from Linux filesystem path.
INFO	No token pause mechanism on PRED	PREDToken.sol	Standard ERC20. No pause. Acceptable for portfolio project.
INFO	No timelock on admin functions	All contracts	Recommend 48-hour timelock for mainnet deployment.

4. Detailed Findings

FINDING-001: Reentrancy Protection

Severity: CRITICAL → MITIGATED

All ETH-transferring functions in PredictionMarket and MultiOutcomeMarket are protected by OpenZeppelin's ReentrancyGuard. The Checks-Effects-Interactions pattern is followed throughout.

In claimReward():

- CHECK: resolved == true, hasClaimed[msg.sender] == false, shares > 0
- EFFECT: hasClaimed[msg.sender] = true (set BEFORE ETH transfer)
- INTERACTION: msg.sender.call{value: payout}("")

Status: RESOLVED — ReentrancyGuard + CEI pattern implemented correctly.

FINDING-002: Centralized Admin Control

Severity: HIGH → PARTIALLY MITIGATED

All contracts use a single-owner pattern (OpenZeppelin Ownable). The owner can:

- Update the oracle address on MarketFactory
- Change fee recipient and fee percentage (up to 5% cap)
- Pause/unpause markets
- Resolve multi-outcome markets via MultiOracle
- Authorize/deauthorize callers in LiquidityMining

Mitigations implemented:

- Fee hard-capped at 5% in the contract — owner cannot extract more
- Fee recipient is per-market immutable — set at deployment only
- Oracle changes only affect future markets, not existing ones

Recommended for mainnet:

- Transfer ownership to a Gnosis Safe multisig (3-of-5 threshold)
- Add a 48-hour timelock on oracle and fee changes

FINDING-003: Oracle Trust Assumption

Severity: HIGH → MITIGATED (for Chainlink markets)

Markets using MockOracle rely on admin honesty for correct resolution. A compromised admin could resolve markets incorrectly, stealing the losing pool from winners.

The ChainlinkOracle mitigates this for price-conditional markets by using Chainlink's decentralized oracle network, which aggregates from 31+ node operators with cryptographic proofs.

For markets requiring subjective resolution (e.g., sports outcomes), admin-controlled resolution remains a trust assumption. Future work: integrate UMA's optimistic oracle for dispute-based resolution.

FINDING-004: Child Contract Deployment Gas

Severity: MEDIUM → MITIGATED

Deploying a child PredictionMarket contract inside createMarket() costs approximately 3,000,000 gas due to the size of the PredictionMarket bytecode. Standard wallet gas estimates (~500,000) fail.

The frontend hardcodes gas: BigInt(3000000) for the createMarket transaction. This was discovered via cast run analysis of a failed transaction.

FINDING-005: Timestamp Manipulation

Severity: LOW → ACCEPTABLE

PredictionMarket uses block.timestamp for expiry checks. Miners (post-merge: validators) can manipulate block.timestamp by up to approximately 12 seconds.

The MIN_EXPIRATION_PERIOD is 1 hour, making a 12-second manipulation negligible. Additionally, a 300-second buffer is added in the frontend at market creation time.

FINDING-006: Integer Overflow/Underflow

Severity: LOW → MITIGATED

All arithmetic uses Solidity 0.8.24 built-in overflow protection. Explicit SafeMath is not required. All division operations check for zero denominator via custom error conditions (winningPool == 0 reverts).

FINDING-007: No Contract Existence Check for Oracle

Severity: INFO

The oracle address is validated for non-zero in the constructor but not checked for contract code existence. Sending ETH to an EOA oracle address would cause calls to succeed (returning 0 bytes) without reverting.

Since oracle address is set by admin at deployment and immutable per-market, the attack surface is limited to admin error. Recommendation: add `address.code.length > 0` check in production.

5. Access Control Matrix

Function	PredictionMarket	MarketFactory	LiquidityMining	ReferralSystem	MultiOracle
Anyone	buyYesShares, buyNoShares, resolve, claimReward	createMarket	claimRewards	claimEarnings, registerReferral*	tryResolve*
Owner Only	pause, unpause, setLiquidityMining	setOracle, setFee, setCollector, pause, deactivate	setAuthorizedCaller, setRewards	setReferralFee, setAuthorizedCaller	resolve, setResolver
Authorized Caller	—	—	recordCreation, recordTrade	recordTrade	—

* registerReferral and tryResolve are callable by anyone but only have effect under specific conditions.

6. Test Coverage

The Foundry test suite contains 130+ tests covering the core protocol:

Test Category	Count	Coverage
Market creation tests	12	All valid/invalid expiry, question, fee combinations
Trading tests	24	YES/NO buys, fee deduction, pool update correctness
Resolution tests	18	YES/NO/INVALID outcomes, payout math verification
Claim tests	16	Winner claims, double-claim prevention, hasClaimed flag
Admin tests	20	Owner functions, pause/unpause, oracle updates
Fuzz tests	40+	Random amounts, random expiries, random outcome sequences
Invariant tests	10+	<code>totalPool == sum(yesShares) + sum(noShares)</code> , fee consistency

Run tests with:

```
cd contracts && forge test -vv
```

7. Security Recommendations

7.1 Before Mainnet Deployment

Priority	Recommendation
P0 — Critical	Commission a formal audit from Certik, Trail of Bits, or Spearbit
P0 — Critical	Deploy all admin functions behind a 3-of-5 Gnosis Safe multisig
P1 — High	Add 48-hour timelock on oracle address and fee changes
P1 — High	Replace MockOracle with UMA optimistic oracle or Chainlink Functions
P2 — Medium	Add address.code.length > 0 check for oracle in constructors
P2 — Medium	Add emergency withdrawal function for stuck ETH (only for INVALID markets)
P3 — Low	Add events for all admin actions (some are already emitted)
P3 — Low	Consider deploying on Base L2 to reduce gas costs

7.2 Monitoring Recommendations

- Monitor factory for unusual createMarket() volume (potential spam)
- Alert on oracle address changes via event monitoring
- Monitor LiquidityMining for unusual PRED mint rates
- Set up Tenderly alerts for large claimReward() calls

8. Known Non-Issues (False Positives)

The following patterns may trigger automated scanners but are intentional design decisions:

- Low-level call for ETH transfer: Required for flexible recipient. Success is checked.
- block.timestamp usage: Acceptable given 1-hour minimum market duration.
- try/catch around LiquidityMining calls: Intentional — reward failure should not block trading.
- msg.sender in constructor: Used for Ownable initialization, standard OpenZeppelin pattern.
- Unbounded arrays in getAllMarkets(): Acceptable for testnet. Pagination recommended for mainnet.

9. Conclusion

The PredictX smart contract system demonstrates solid security fundamentals for a portfolio project:

- Reentrancy protection applied correctly throughout
- Checks-Effects-Interactions pattern consistently followed
- Access control clearly defined with minimal privilege escalation
- Fee parameters hard-capped to prevent admin abuse
- Chainlink integration uses battle-tested aggregator pattern

The primary risk vectors remaining are centralized admin control and unaudited code. Both are acceptable for a Sepolia testnet portfolio project but **MUST** be addressed before any mainnet deployment with real user funds.

Overall security posture: **ACCEPTABLE FOR TESTNET, NOT READY FOR MAINNET** without a formal audit and multisig governance.

Appendix A — Solidity Vulnerability References

SWC ID	Vulnerability	Status in PredictX
SWC-107	Reentrancy	MITIGATED — ReentrancyGuard + CEI
SWC-116	Timestamp Dependence	LOW RISK — 1hr minimum duration
SWC-101	Integer Overflow	MITIGATED — Solidity 0.8.24 built-in
SWC-115	Authorization via tx.origin	NOT PRESENT — uses msg.sender throughout
SWC-104	Unchecked Return Values	MITIGATED — all ETH transfers checked
SWC-106	Unprotected SELFDESTRUCT	NOT PRESENT — no selfdestruct
SWC-111	Use of deprecated functions	NOT PRESENT — no .transfer() or .send()
SWC-135	Code Size Check Bypass	LOW RISK — oracle EOA risk documented